計算物理学II

第2講：コマンド操作とデータの可視化

秋山 進一郎

2024年10月11日

今日の授業の目標

・引き続き,Terminalの操作に慣れていく

・コマンド操作

・図の作成（gnuplot）

毎回の授業の流れ

“compphy2”に移動する

↓

“git fetch”と“git merge”で演習資料を入手

↓

“lecture*”に移動し, 演習開始

本日の演習内容

- コマンドを使ったファイル操作について
  - 前回の復習といくつかの新しいコマンド（cp, mv, rm…）
- Gnuplotによる図の作成
  - 関数のプロット
  - 数値データのプロット
  - スクリプトの活用

ブランチを切る

・原則, Gitではmainブランチで作業しない

・“exercise”という名前のブランチを作成（①）

・mainからexerciseへの切り替え（②）

・自分がいるブランチの確認（③）

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git branch exercise   ①
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git switch exercise   ②
Switched to branch 'exercise'
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git branch            ③
* exercise
  main
```

前回学んだコマンドを使って以下を実行しましょう

- “exercise_command”というディレクトリを作る

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ mkdir exercise_command
```

- “exercise_command”へ移動

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ cd exercise_command/
akiyama.shinichir.ga@icho01:~/compphys2/lecture2/exercise_command$
```

- “test.txt”というファイルを作成

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2/exercise_command$ touch test.txt
```

- “test.txt”を好きなエディタ（emacs, VS code, vi, …）を使って編集してみよう
  - 「test」と入力して保存し, エディタを閉じる
- 編集内容の確認

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2/exercise_command$ cat test.txt
test
```

ファイルのコピー：cpコマンドを使ってみる

- **ファイルのコピーはcpコマンドで行う**
  - “test.txt”を“cp_test.txt”という名前でコピーする
  - コマンドの構造は, “cp *コピーしたいファイル（のパス）* *コピー先のファイル（のパス）*”

    スペース
  - パスの説明は後で. まずは以下のコマンドを実行してみましょう

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2/exercise_command$ cp ./test.txt ./cp_test.txt
```

  - lsを使って“cp_test.txt”が作成されたことを確認せよ
  - catを使って“cp_test.txt”の中身を確認し, コピーされていることを確認せよ

ファイルのコピー：cpコマンドを使ってみる

- **ファイルのコピーはcpコマンドで行う**
  - “./”はカレントディレクトリを意味する
  - “./test.txt”は「カレントディレクトリの中のtest.txt」を意味する
  - カレントディレクトリ内での操作では“./”は省略可能
    - 試しに以下のコマンドも実行せよ

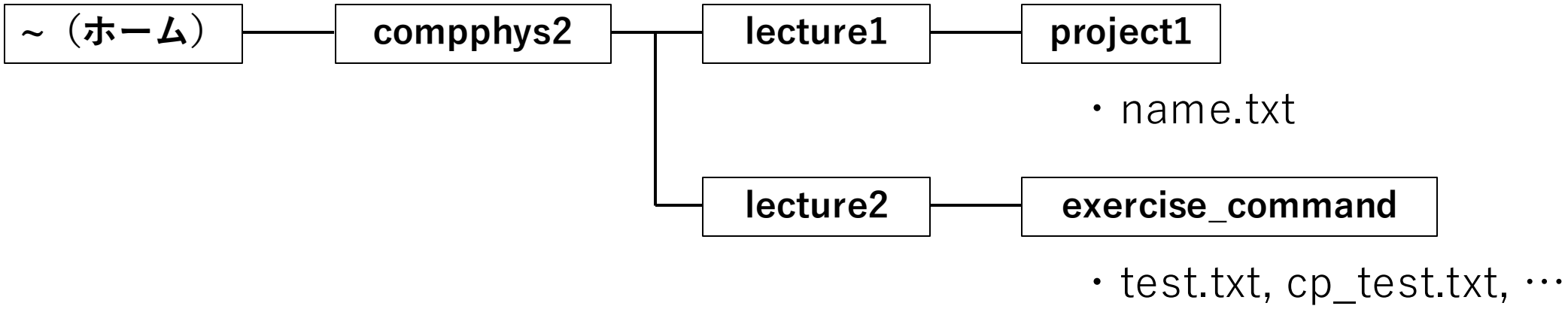
```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2/exercise_command$ cp test.txt cp_cp_test.txt
```

  - lsを使って“cp_cp_test.txt”が作成されたことを確認せよ
  - catを使って“cp_cp_test.txt”の中身を確認し, コピーされていることを確認せよ

ディレクトリを跨ぐファイルのコピー

・前回作成した“name.txt”を今いるディレクトリにコピーしよう

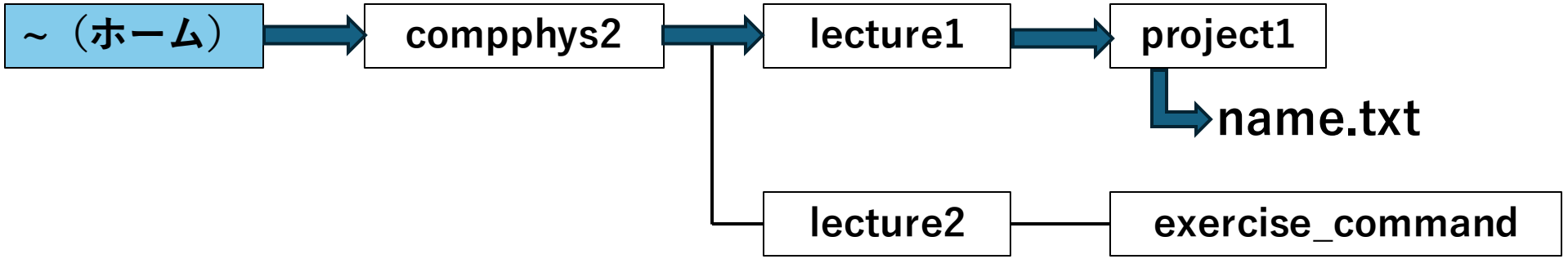
・現状のディレクトリ構造は以下のようになっている



方法1：絶対パスを使う

・先ほど出てきた「パス」には二種類ある（絶対パスと相対パス）

・絶対パスとは, **ディレクトリのツリー構造のトップを起点**とした記述形式



・test.txt, cp_test.txt, …

・絶対パスによるname.txtの記述

・“~/compphys2/lecture1/project1/name.txt”

・「ホームの中のcompphys2の中のlecture1の中のproject1の中のname.txt」という意味である

方法1：絶対パスを使う

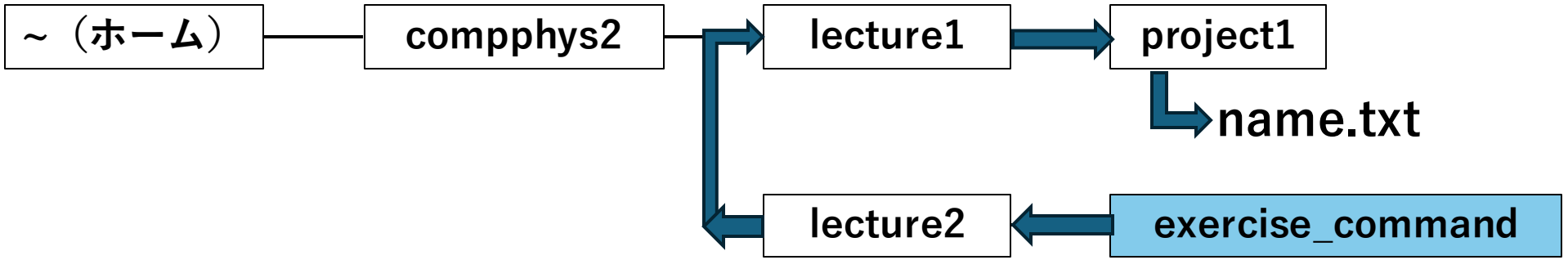
- “cp *コピーしたいファイルのパス* *コピー先のファイルのパス*”でコピーする
  - コピー後のファイル名はname.txtのままとする
  - コピー後のファイル名を変えない場合, *コピー先のファイルのパス*の部分は, 単にカレントディレクトリ（./）とする

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2/exercise_command$ cp ~/compphys2/lecture1/project1/name.txt ./
```

  - lsを使って“name.txt”がカレントディレクトリにコピーされたことを確認せよ
  - catを使って“name.txt”の中身を確認し, 正しくコピーされていることを確認せよ

方法2：相対パスを使う

・相対パスとは, **カレントディレクトリを起点**とした記述形式



・test.txt, cp_test.txt, …

・相対パスによるname.txtの記述

・“../../lecture1/project1/name.txt”

・“../”はカレントディレクトリから見て一つ上のディレクトリ（lecture2）

・“../../”はカレントディレクトリから見て二つ上のディレクトリ（compphy2）

・「二つ上のディレクトリの中のlecture1の中のproject1の中のname.txt」という意味

方法2：相対パスを使う

- “cp *コピーしたいファイルのパス* *コピー先のファイルのパス*”でコピーする
  - コピー後のファイル名はcp_name.txtとする

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2/exercise_command$ cp ../../lecture1/project1/name.txt ./cp_name.txt
```

  - lsを使って“cp_name.txt”がカレントディレクトリにコピーされたことを確認せよ
  - catを使って“cp_name.txt”の中身を確認し, 正しくコピーされていることを確認せよ

絶対パスと相対パスの使い分け

- 絶対パスはカレントディレクトリに**依存しない**記述形式
  - いわば各ディレクトリや各ファイルの住所
  - リンクを貼る時には便利
- 相対パスはカレントディレクトリに**依存した**記述形式
  - 実際の作業を進める時には相対パスをよく使う

カレントディレクトリの絶対パスの取得：pwdを使う

・カレントディレクトリの絶対パスを知りたい場合はpwdコマンドを使う

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2/exercise_command$ pwd
/home/akiyama.shinichir.ga/compphys2/lecture2/exercise_command
```

・ここではホームディレクトリは“~/”ではなく“/home/”で表示されれている

・pwdで自分の居場所を確認できる

ディレクトリのコピー：cpコマンドを使う

・今度は, 前回作成した“project1”（ディレクトリ）を今いるディレクトリにコピーしよう

・相対パスによるコピー

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2/exercise_command$ cp ../../lecture1/project1 ./
```

・このコマンドを実行すると…

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2/exercise_command$ cp ../../lecture1/project1 ./
cp: -r not specified; omitting directory '../../lecture1/project1'
```

・lsを使ってproject1がコピーされているかどうかを見てみよう

・Terminalは何と言っているでしょうか？（次ページ）

ディレクトリのコピー：cpコマンドを使う

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2/exercise_command$ cp ../../lecture1/project1 ./
cp: -r not specified; omitting directory '../../lecture1/project1'
```

・「-rがない」と言っている

・ディレクトリをコピーする方法

・“cp **–r** *コピーしたいディレクトリのパス* *コピー先のディレクトリのパス*”

・素朴に下記のコマンドを打っても良いが, ↑**キーを一回押すと直前に打ったコマンドが出てくる.** そこに-rを書き加えても良い

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2/exercise_command$ cp -r ../../lecture1/project1 ./
```

・lsを使ってproject1がコピーされているかどうかを見てみよう

コマンドに対するオプション

- コマンドには様々な**オプション**が用意されている
  - オプションを指定することでコマンドの機能が拡張される
- 先ほどの場合だと, “-r”がcpコマンドに対するオプション
- 各コマンドに用意されているオプションは“コマンド --help”で閲覧できる
  - 以下のコマンドを実行し, cpにどのようなオプションがあるか見てみよう

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2/exercise_command$ cp --help
```

その他のよく使うオプション一例

・lsコマンド

・一度ホームディレクトリに戻りましょう（その場でcdする）

・ホームディレクトリでlsを実行

```
akiyama.shinichir.ga@icho01:~$ ls
 AppData   Contacts   Documents   Favorites   Music     'Saved Games'  'Start Menu'  Videos    WinSCP.ini   wwws          テンプレート   ビデオ     ミュージック
 Browser   Desktop    Downloads   Links       Pictures   Searches       Temp         WINDOWS   compphys2    ダウンロード   ドキュメント   ピクチャ   公開
```

・今度はlsに**-la**オプションをつけて実行してみましょう

```
akiyama.shinichir.ga@icho01:~$ ls -la
```

・表示形式が変わり, lsでは見えていなかったディレクトリやファイルが見えたらOK

・-laは, -l（詳細情報表示）と-a（隠しファイルを含む全てを表示）というオプションを合わせたもの. よく使う

・隠しファイルとは名前が“.”から始まるもの

ファイルの移動：mvコマンドを使う

・まずホームから“exercise_command”のディレクトリまで移動しましょう

・“cp_cp_test.txt”をlecture2ディレクトリ直下へ移動させましょう

- **ファイルを移動するにはmvコマンドを使う**
  - コマンドの構造は, “mv *移動したいファイルのパス* *移動先のファイルのパス*”

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2/exercise_command$ mv ./cp_cp_test.txt ../
```

  - 「カレントのcp_cp_test.txtを一つ上のディレクトリへ移動する」の意味
  - lsを使って, exercise_commandの中にcp_cp_test.txtが**無い**ことを確認せよ
  - lecture2ディレクトリに移動してlsせよ. cp_cp_test.txtがあるはず
  - cpとmvの違いに注意しましょう

ファイル名の変更：mvコマンドを使う

・mvコマンドを応用するとファイル名の変更が可能

・lecture2ディレクトリに移動させた“cp_cp_test.txt”を“test.txt”という名前に変更してみましょう

・**ファイル名の変更とはファイルの移動の一種**

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ mv ./cp_cp_test.txt ./test.txt
```

・lsを使って, cp_cp_test.txtがtest.txtになったことを確認せよ

ディレクトリの移動：mvコマンドを使う

- 今度は, “exercise_command”の中にある“project1”ディレクトリを“lecture2”直下へ移動させてみましょう
  - 相対パスを使えば, わざわざ“exercise_command”に入らずに次のようにできます

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ mv ./exercise_command/project1 ./
```

  - lsを使って, project1がlecture2の中に移動されたことを確認せよ
  - cpでディレクトリのパスを扱うには-rオプションが必要だったが, mvでは不要

ファイルやディレクトリの削除：rmコマンドを使う

- **ファイルやディレクトリの削除は, rmコマンドを使う**
  - **rmで削除すると元には戻せないので, 実行時は要注意！**
- lecture2直下にあるtest.txtを削除してみます
  - -iオプション：削除前に本当に削除して良いか確認する
  - 次のコマンドを実行

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ rm -i test.txt
rm: 通常ファイル 'test.txt' を削除しますか？ y
```

  - Terminalが削除していいか確認してくる
    - 削除する場合: yを押してEnterを押す（今回はこちら）
    - 削除する場合: nを押してEnterを押す
  - lsでtest.txtが消えたことを確認せよ

ファイルやディレクトリの削除：rmコマンドを使う

- **ディレクトリの削除は, rmコマンドに-rオプションをつける**
  - -iオプションと組み合わせる場合は-riとする（-irでもOK）
- lecture2直下にあるproject1を削除してみます
  - 次のコマンドを実行

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ rm -ir projoct1/
rm: ディレクトリ 'projoct1/' を削除しますか？ y
```

  - Terminalが削除していいか確認してくる
    - ディレクトリ配下に入るか尋ねてきた場合もyを押してEnter
    - その場合, 配下のファイルを削除するか逐一尋ねられるので, 逐一yを押してEnter
    - 配下のファイルが全て削除されたら, 最後にディレクトリを削除するか尋ねてくるので, yを押してEnter
  - lsでproject1が消えたことを確認せよ

本日の演習内容

- コマンドを使ったファイル操作について
  - 前回の復習といくつかの新しいコマンド（cp, mv, rm…）
- Gnuplotによる図の作成
  - 関数のプロット
  - 数値データのプロット
  - スクリプトの活用

Gnuplotとは？

- 入力された数式やデータをグラフ化してくれるソフトウェア
- 二種類の形式（使い方）がある
  - 対話型
  - バッチ処理形式型（推奨）
- 公式のUser manualはここから
  - User manualは300ページ以上ある
  - 知りたい情報があれば, ページ内検索で拾う

対話型

- gnuplotコマンドで起動
  - 一行ずつgnuplotに命令を出していく
  - あまり使わない
    - 理由は後で

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ gnuplot

        G N U P L O T
        Version 5.2 patchlevel 8    last modified 2019-12-01

        Copyright (C) 1986-1993, 1998, 2004, 2007-2019
        Thomas Williams, Colin Kelley and many others

        gnuplot home:     http://www.gnuplot.info
        faq, bugs, etc:   type "help FAQ"
        immediate help:   type "help"  (plot window: hit 'h')

Terminal type is now 'qt'
gnuplot>
```

- 例えば, “plot x**2”や“plot sin(x)”と打ってEnterを押してみましょう
  - 図が出てくればOK. 確認後, 図は閉じてよい
- “exit”と打ってEnterを押すとTerminalに戻ってくる
  - Terminalに戻りましょう

バッチ処理形式

- gnuplotへのコマンドをリストしたファイルを用意し, それを読み込ませて図を作成する
  - このようなファイルのことを**スクリプト**と呼ぶ
  - スクリプトは普通の.txtファイルでもいいが, gnuplotのスクリプトであることを明確にする目的で, .gplや.pltもよく使われる
- まず, lecture2の直下に“exercise_gnuplot”というディレクトリを作り, そこへ移動しましょう
- lecture2/script/sample.gplがサンプルとして用意したgnuplotのスクリプトです
  - “sample.gpl”を“exercise_gnuplot”へコピーしましょう

演習で使う数値データの説明

- lecture2/data/population.datがこれから可視化する数値データです
  - “population.dat”も“exercise_gnuplot”へコピーしましょう
- “population.dat”には, 大正9年から平成27年までの全国, 及び各都道府県の人口が格納されています（５年ごと）
  - “population.dat”は, ここからダウンロードできる国勢調査のオープンデータに基づいています
- catを使い, “population.dat”を見てみましょう

出典：「政府統計の総合窓口(e-Stat)」、調査項目を調べる－国勢調査 「男女別人口－全国，都道府県（大正９年～平成27年）」

演習で使う数値データの説明

・“population.dat”の構造は次のようになっています

・1列目：都道府県コード

・2列目：都道府県名

・3列目：元号

・4列目：和暦（年）

・5列目：西暦（年）

・6列目：人口（総数）

・7列目：人口（男性）

・8列目：人口（女性）

```
akiyama.shinichir.ga@icho01:~/compphys2$ cat ./lecture2/data/population.dat
0       Japan   Taisho  9       1920            55963053        28044185        27918868
1       Hokkaido        Taisho  9       1920            2359183 1244322 1114861
2       Aomori  Taisho  9       1920            756454  381293  375161
3       Iwate   Taisho  9       1920            845540  421069  424471
4       Miyagi  Taisho  9       1920            961768  485309  476459
5       Akita   Taisho  9       1920            898537  453682  444855
6       Yamagata        Taisho  9       1920            968925  478328  490597
```

出典：「政府統計の総合窓口(e-Stat)」、調査項目を調べる－国勢調査 「男女別人口－全国，都道府県（大正９年～平成27年）」

特定の数値データを抜き出す

- “population.dat”から特定の情報を抜き出し, gnuplotで可視化してみましょう
  - **例題：「横軸に西暦, 縦軸に日本の総人口を取って, 総人口の推移を見よう」**
- ここで, Terminalから使える便利なコマンドを紹介します
  - **grep**コマンド＝あるファイルから特定の文字列を含んだ行だけを抜き出す
  - “grep ‘*抜き出したい文字列*‘ *ファイル（のパス）*”
- 以下のコマンドを実行してみましょう

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2/exercise_gnuplot$ grep 'Japan' ./population.dat
```

- Japanを含む行だけ抜き出されたらOK

特定の数値データを抜き出し, 保存する

・さらに, 次のようにgrepコマンドを使うと, 抜き出した結果を新たに保存できます

・“grep ‘*抜き出したい文字列*‘ *ファイル（のパス）* >> *保存ファイル（のパス）*”

・以下のコマンドを実行してみましょう（↑キーを押すと便利）

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2/exercise_gnuplot$ grep 'Japan' ./population.dat >> ./japan.dat
```

・catを使い, “japan.dat”を見てみよう

スクリプトを使ってgnuplotに図を書かせる

- 先ほどcpした“sample.gpl”をcatで見てみましょう
  - 各行の意味は#以下で説明されている
  - #以下はコマンドとして読み込まれない（コメントアウト）
  - 各行の意味を確認したら次のページへ

```
reset
set terminal pdfcairo color enhanced font "Helvatica,18"        #図の出力形式を指定
set output "output_file_name.pdf"                                #出力ファイル名を指定
set title font "Times,14"                                        #タイトルのフォント,大きさを指定
set title "Title"                                                #タイトルを指定
set key font "Times,12"                                          #凡例(key)のフォント,大きさを指定
set key top left                                                 #凡例(key)の表示場所を指定
set xlabel "x label"                                             #x軸の名前（見出し）を指定
set ylabel "y label"                                             #y軸の名前（見出し）を指定
set xlabel font "Times,14"                                       #x軸の見出しのフォント,大きさを指定
set ylabel font "Times,14"                                       #y軸の見出しのフォント,大きさを指定
set xtics font "Times,14"                                        #x軸の目盛りのフォント,大きさを指定
set ytics font "Times,14"                                        #x軸の目盛りのフォント,大きさを指定
#set xrange [0:1]                                                #x軸の範囲を指定（指定しない場合は読み込んだ数値データに自動的に合わされる）
#set yrange [0:1]                                                #y軸の範囲を指定（指定しない場合は読み込んだ数値データに自動的に合わされる）
#set xtics 1                                                     #x軸の目盛りの幅を指定
#set ytics 1                                                     #y軸の目盛りの幅を指定
#set format x "%0.001f"                                          #x軸の目盛りの書式を指定
#set format y "%0.1f"                                            #y軸の目盛りの書式を指定
plot "input_file_name" using x:y pt 1 lc 1 w p title "key"       #"input_file_name"のx列目をx軸，y列目をy軸とし，シンボルパターン1，色1，点を使い，凡例を"key"して図を作成

set terminal pop
```

スクリプトを編集する

- “plot”の行を編集していく
  - **例題：「横軸に西暦, 縦軸に日本の総人口を取って, 総人口の推移を見よう」**
  - 読み込むファイルは, “japan.dat”
  - x軸に使うのは5列目, y軸に使うのは6列目

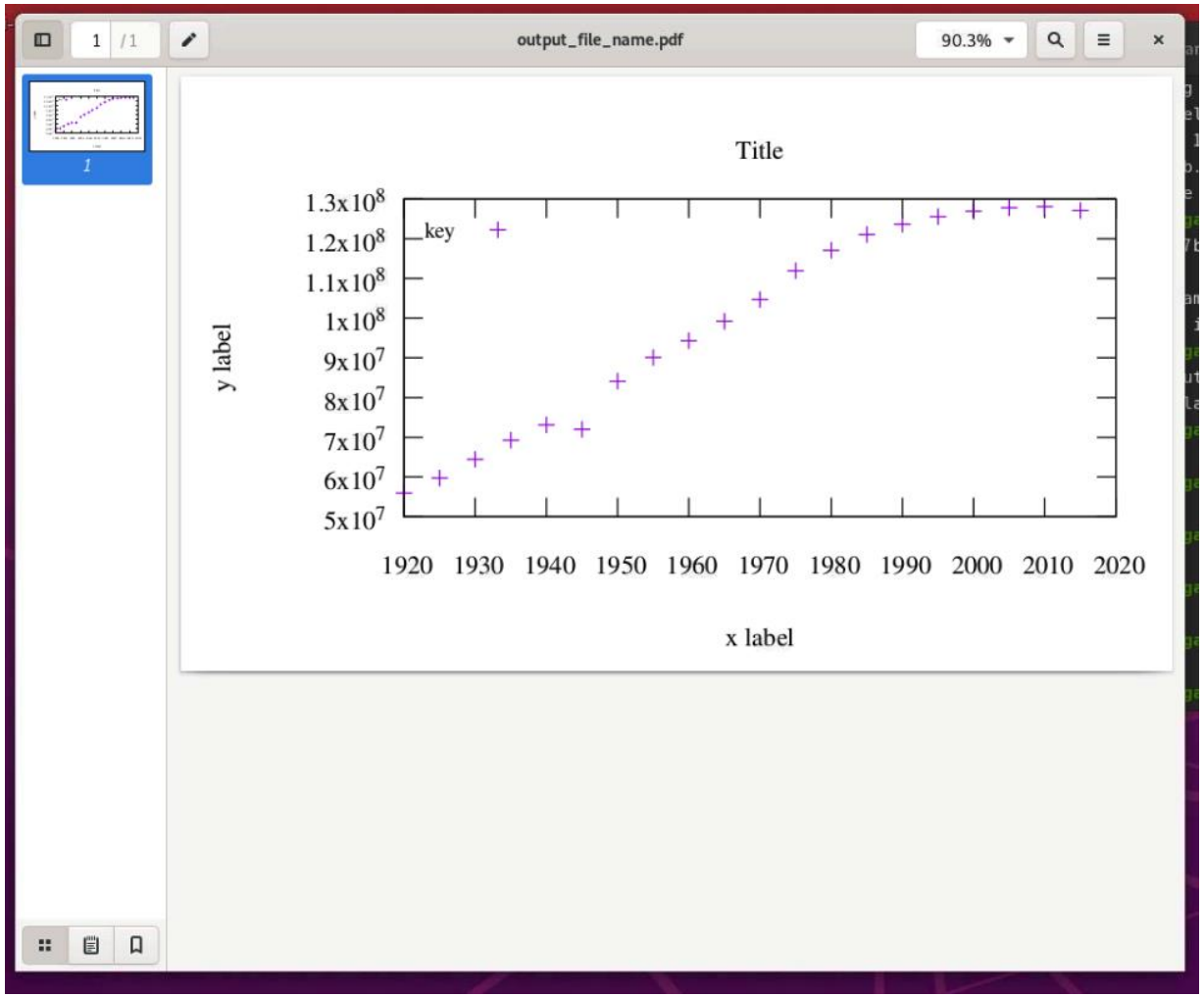
```
plot "japan.dat" using 5:6 pt 1 lc 1 w p title "key"
```

- “plot”の行を上のように編集したら, gnuplotにスリプトを読ませる
  - “gnuplot *スクリプトファイル*”

```
akiyama.shinichir.ga@kiri:~/compphys2/lecture2/exercise_gnuplot$ gnuplot sample.gpl
```

作成した図の確認

- “output_file_name.pdf”というファイルが出来たはず
  - lsで確認せよ
  - xdg-openで図を開いてみよ
  - 右のような図ができましたか？



バッチ処理形式が推奨される理由

- **スクリプトを見返せば, 図の作成に用いた数値データ（ファイル）が何だったか分かる**
  - 対話型ではこうした記録は残らない
  - 「この図は何をプロットしたんだっけ？」という事態を防ぐことができる
  - **図の再現性を担保することにつながる**

図を改善していきましょう

- “sample.gpl”を適宜編集し, 図の見栄えを改善しましょう
  - “Title”, “x label” , “y label” , “key”に相応しい名前をつけましょう
  - 出力ファイル名を変更しましょう（何の図か分かるような名前をつけましょう）
  - 図のシンボルや色は整数値で指定されます
    - 違う整数値に変えてみましょう
  - 総人口に加えて, 男性人口（7列目）, 女性人口（8列目）の推移も可視化してみましょう
    - plot以下の内容をカンマ（,）で区切りながら付け足していけばOK

```
plot "japan.dat" using 5:6 pt 1 lc 1 w p title "key", "japan.dat" using 5:7 pt 2 lc 2 w p title "key",
```

    - “key”にも相応しい名前をつけましょう

図を改善していきましょう

- “sample.gpl”を適宜編集し, 図の見栄えを改善しましょう
  - 配布したスクリプトではいくつかの行がコメントアウトしてあります
  - コメントアウトを外して（“set”直前の#を削除して）軸の書式を変えてみましょう

ここまで終わったら

- お疲れ様でした
- 時間に余裕のある人は, “population.dat”から好きな都道府県データを抜き出し, 人口の推移を可視化してみましょう
- [宿題] gnuplotで使用可能なシンボル, 色と整数値の対応をネットで調べてみる

作業が終わったら作業用ブランチを消す

- 実際のプログラム開発では, 作業用ブランチで新たに作成したファイル等もGitの管理下に置くが, 今日の講義ではやりません
  - mainブランチに戻る（①）
  - exerciseブランチを削除（②）
  - 自分がいるブランチの確認（③）

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git branch -d exercise
Deleted branch exercise (was 2bf0644).
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git branch
* main
```

①

②

③